SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE CODE: CSA1409 **COURSE NAME: Compiler Design for Higher Level Processing**

COURSE OUTCOME COVERED

CO2: Construct top-down and bottom up parsers using CFG for parsing conflicts and error recovery for efficient syntax tree generation. *(BL3)*

Assessment Tool Weightage: 40%

QUESTIONS:4 **Total Marks:50**

Annexure A

DESIGN TASK

Code of Conduct:

*I **E.Ragul / 192571032** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:

E.Ragul

Course Faculty

Annexure A

DESIGN TASK

Q1. Predictive Parser Design (Application Level)

Task:
Construct an LL(1) predictive parsing table for the given grammar and demonstrate its application by parsing the specified input string step-by-step, clearly showing stack operations, input symbols, and parsing actions.

.

Grammar:
E → T E'
E' → + T E' | ε
T → F T'
T' → * F T' | ε
F → (E) | id

Conditions:

- Remove left recursion (if any)
- Compute FIRST and FOLLOW sets
- Construct LL(1) parsing table

Test Case:
Input String:
id + id * id
Expected Output:

- Step-by-step parsing actions
- Final result: **String Accepted**

Q2. Shift-Reduce Parsing (Application Level)

Task:
Apply shift-reduce parsing to the given grammar and input string, and demonstrate the complete parsing process by showing each step with stack contents, remaining input, and parsing actions (shift, reduce, accept, or error), clearly indicating all reductions and decisions made during parsing.

Grammar:
E → E + E
E → E * E
E → id

Conditions:

- Show stack, input buffer, and action at each step
- Resolve conflicts using precedence (* has higher precedence than +*)

Test Case:
Input:
id + id * id
Expected Output:

- Sequence of actions: Shift / Reduce / Accept
- Final result: **String Accepted**

Integrated Design Question (SLR, LALR, CLR Parsing)

Q12. Develop a New Parsing Algorithm

Task:
Propose and design a new parsing algorithm that improves efficiency or readability compared to existing LL and LR techniques.

Conditions:

- Define algorithm steps clearly
- Compare with LL and LR methods
- Analyze time and space complexity
- Demonstrate parsing on sample grammar
- Highlight improvements and trade-offs
- Provide pseudo-code implementation

Test Case:
Input: id * id + id

Answer:

A new parsing algorithm called Hybrid Predictive Operator Parser (HPOP) is proposed to improve the readability and efficiency of traditional LL and LR parsing techniques. It combines predictive parsing with operator-precedence handling to parse arithmetic expressions and generate syntax trees efficiently.

Proposed Algorithm

The algorithm identifies operands using predictive parsing and processes operators according to their precedence. Multiplication and division have higher precedence than addition and subtraction.

Algorithm steps

1. Read the input expression and convert it into tokens.
2. Identify identifiers, operators, and parentheses.
3. Parse operands using the parseFactor() function.
4. Parse multiplication operations using the parseTerm() function.
5. Parse addition operations using the parseExpression() function.
6. Construct syntax-tree nodes for each recognized operator.
7. Check whether the complete input is consumed.
8. Report syntax errors when unexpected tokens are encountered.

Sample Grammar

$$E \rightarrow E + T \mid T$$
$$T \rightarrow T * F \mid F$$
$$F \rightarrow (E) \mid id$$

$$
\begin{aligned}
E &\to TE' \\
E' &\to +TE' \mid \varepsilon \\
T &\to FT' \\
T' &\to *FT' \mid \varepsilon \\
F &\to (E) \mid id
\end{aligned}
$$

Parsing Demonstration

Input: id * id + id

Step	Input processed	Parsing action
1	id	Recognize the first operand
2	id * id	Recognize multiplication
3	id * id + id	Recognize addition
4	End of input	Accept the expression

Pseudo-Code Implementation

```
parseExpression():
    left = parseTerm()
    while nextToken == '+':
        consume('+')
        right = parseTerm()
        left = makeNode('+', left, right)
    return left
parseTerm():
    left = parseFactor()
    while nextToken == '*':
        consume('*')
        right = parseFactor()
        left = makeNode('*', left, right)
    return left
parseFactor():
    if nextToken == 'id':
```

```
        return makeLeaf(consume('id'))
    if nextToken == '(':
        consume('(')
        node = parseExpression()
        consume(')')
        return node
    reportSyntaxError()
```

Comparison with LL and LR Parsing

Feature	LL Parser	LR Parser	Proposed HPOP
Parsing approach	Top-down	Bottom-up	Hybrid approach
Left recursion	Must be removed	Supported	Avoided through iterative parsing
Complexity of implementation	Low	High	Moderate
Operator precedence	Grammar-based	Grammar and parsing tables	Explicit precedence through parsing functions
Syntax-tree generation	During production expansion	During reductions	During operator processing
Error detection	Predictive checks	State-based checks	Expected-token checks

Improvements and Trade-Offs

Improvements:

- Provides a simple and readable parsing structure.
- Preserves operator precedence and associativity.
- Constructs the syntax tree during parsing.
- Processes arithmetic expressions in linear time.

Trade-offs:

- Best suited to expression grammars rather than arbitrary programming-language grammars.
- New operators require suitable precedence and associativity rules.
- General-purpose LR parsers support a wider range of grammars

Conclusion

The proposed Hybrid Predictive Operator Parser successfully parses id * id + id and constructs the correct syntax tree. It combines predictive parsing with explicit operator-precedence handling to provide a readable and efficient solution for arithmetic expressions.

Q13. Design a Compiler Front-End for Expression Language

Task:
Create a complete front-end design for a simple expression language including lexical analyzer, parser, and intermediate code generator.

Conditions:

- Define token specifications
- Design grammar and parser (LL/LR)
- Generate syntax tree
- Produce three-address code
- Handle syntax errors
- Demonstrate full compilation process

Test Case:
Input: a = b + c * d

ANSWER:

A compiler front-end converts a source program into an intermediate representation by performing lexical analysis, syntax analysis, syntax-tree construction, and intermediate code generation. It also detects and reports syntax errors.

Token Specifications

The lexical analyzer reads the source program and converts it into tokens.

Token	Description	Example
ID	Identifier	a, b, total
ASSIGN	Assignment operator	=
PLUS	Addition operator	+
MUL	Multiplication operator	*
NUM	Numeric constant	10, 25
LPAREN	Left parenthesis	(
RPAREN	Right parenthesis	)
WS	Whitespace	Space or tab

For the input a = b + c * d, the generated tokens are:

ID(a), ASSIGN, ID(b), PLUS, ID(c), MUL, ID(d)

Grammar Design

The following context-free grammar (CFG) defines the expression language:

$$
\begin{aligned}
S &\rightarrow id = E \\
E &\rightarrow E + T \mid T \\
T &\rightarrow T * F \mid F \\
F &\rightarrow (E) \mid id \mid num
\end{aligned}
$$

Here, \(S\) represents an assignment statement, \(E\) represents an expression, \(T\) represents a term, and \(F\) represents a factor.

For an LL parser, left recursion is eliminated:

$$
\begin{aligned}
S &\rightarrow id = E \\
E &\rightarrow TE' \\
E' &\rightarrow +TE' \mid \varepsilon \\
T &\rightarrow FT' \\
T' &\rightarrow *FT' \mid \varepsilon \\
F &\rightarrow (E) \mid id \mid num
\end{aligned}
$$

This grammar gives multiplication higher precedence than addition.

Parser Design

The LL(1) predictive parser uses a parsing stack, an input buffer, and a parsing table to recognize valid expressions.

Algorithm steps:

1. Read the input source program.
2. Convert the input into tokens using the lexical analyzer.
3. Initialize the parser with the start symbol.
4. Match terminals and expand non-terminals according to the grammar.
5. Construct syntax-tree nodes while parsing.
6. Accept the expression if all tokens are consumed successfully.
7. Report syntax errors if the input does not match the grammar.

Syntax Tree Generation

Input: a = b + c * d

The parser constructs the following abstract syntax tree (AST):

The tree shows that c * d is evaluated before b + (c * d).

Three-Address Code Generation

Three-address code (TAC) is an intermediate representation in which each instruction contains at most one main operation.

For the input a = b + c * d, the generated code is:

t1 = c * d

t2 = b + t1

a = t2

Explanation:

1. t1 = c * d — multiplies c and d.
2. t2 = b + t1 — adds b to the multiplication result.
3. a = t2 — assigns the final result to a.

Here, t1 and t2 are temporary variables.

Syntax Error Handling

The parser detects invalid expressions and reports appropriate error messages.

Incorrect input	Error detected	Recovery method
a = b + * d	Missing operand	Skip the unexpected token and resume parsing where possible
a b + c	Missing assignment operator	Report the expected =
a = b + c *	Missing operand after *	Report the expected identifier, number, or (
a = (b + c	Missing closing parenthesis	Report the expected)

A simple recovery technique is panic-mode error recovery, which skips tokens until a synchronization token such as ; or the end of input is found.

Complete Compilation Process

For the test input a = b + c * d, the front-end performs these operations:

Phase	**Output**
Source program	**a = b + c * d**
Lexical analysis	**Token stream**
Syntax analysis	**Valid assignment statement**
AST generation	**Assignment tree with + and * nodes**
Intermediate code generation	**Three-address code**
Error checking	**No syntax errors**

Time and Space Complexity

Let n be the number of tokens in the input.

- Lexical analysis: $O(n)$.
- LL(1) syntax analysis: $O(n)$ for this grammar.
- Syntax-tree construction: $O(n)$.
- Three-address code generation: $O(n)$.
- Overall time complexity: $O(n)$.
- Overall space complexity: $O(n)$, for token storage, the syntax tree, and intermediate code.

Advantages

1. Converts source code into a structured representation.
2. Maintains correct operator precedence.
3. Generates simple intermediate instructions.
4. Detects and reports syntax errors.
5. Provides a foundation for later compiler phases.

Conclusion

The compiler front-end successfully processes a = b + c * d through lexical analysis, syntax analysis, AST generation, and three-address code generation. It preserves operator precedence, detects syntax errors, and produces an intermediate representation suitable for further compiler processing.

Q14. Create a PEG-Based Parsing System

Task:
Design a parsing system using Parsing Expression Grammars (PEG) for a programming language and evaluate its performance against CFG-based parsers.

Conditions:

- Convert CFG to PEG format
- Handle ordered choice and backtracking
- Implement parsing logic
- Compare ambiguity handling
- Analyze performance and limitations
- Demonstrate parsing process

Test Case:
Input: id + id * id

ANSWER

To design a Parsing Expression Grammar (PEG) parser for an expression language, demonstrate ordered choice and backtracking, and evaluate its performance against CFG-based parsers.

PEG Grammar Design
A Parsing Expression Grammar (PEG) is a formal grammar used to describe the syntax of programming languages. Unlike a Context-Free Grammar (CFG), PEG uses **ordered choice** to select the first successful alternative.

Given CFG:

$$E \rightarrow E + T \mid T$$
$$T \rightarrow T * F \mid F$$
$$F \rightarrow id$$

The grammar contains left recursion, so it must be rewritten for a conventional recursive-descent PEG parser.

Equivalent PEG Grammar:
Start <- Expr EOF
Expr <- Term ('+' Term)*
Term <- Factor ('*' Factor)*
Factor <- '(' Expr ')' / ID
ID <- [a-zA-Z_][a-zA-Z0-9_]*
EOF <- !.

Explanation:

- <- defines a grammar rule.
- / represents ordered choice.
- * means zero or more repetitions.
- ! represents negative lookahead.
- ID matches an identifier.
- EOF ensures that the entire input is consumed.

This grammar gives multiplication higher precedence than addition

Parsing Algorithm

1. Read the input expression.
2. Convert the input into tokens.
3. Start parsing with the Start rule.
4. Parse factors and parenthesized expressions using Factor.
5. Parse multiplication using Term.
6. Parse addition using Expr.
7. Use ordered choice to select between alternatives.
8. Backtrack to a saved input position when a parsing alternative fails.
9. Construct syntax-tree nodes for operators and operands.
10. Accept if the entire input is consumed; otherwise, report a syntax error.

Pseudocode Implementation

```
parseExpr():
   node = parseTerm()

   while nextToken == '+':
      consume('+')
      right = parseTerm()
      node = makeNode('+', node, right)
   return node
parseTerm():
   node = parseFactor()
   while nextToken == '*':
      consume('*')
      right = parseFactor()
      node = makeNode('*', node, right)

   return node

parseFactor():
   if nextToken == ID:
      return makeLeaf(consume(ID))

   if nextToken == '(':
      consume('(')
      node = parseExpr()

      if nextToken != ')':
         reportError("Expected )")

      consume(')')
      return node

   reportError("Expected identifier or (")
   return FAILURE
```

This is a simplified recursive-descent implementation of the PEG expression rules. A complete PEG engine must also implement ordered-choice evaluation and input-position management.

Parsing Demonstration
Input: id + id * id

Step	Input processed	Action
1	id	Recognize the first factor
2	id +	Recognize the addition operator
3	id + id	Recognize the second operand
4	id + id *	Recognize multiplication
5	id + id * id	Construct the multiplication node
6	End of input	Construct the addition node and accept

Syntax Tree:

```
      +
     / \
   id   *
       / \
      id  id
```

The resulting expression is:
[
id+(id*id)
]
Multiplication is evaluated before addition.

Ordered Choice and Backtracking
PEG uses ordered choice to select between alternatives.
Consider the grammar:
S <- 'a' / 'ab'
For input ab, the parser tries 'a' first. It matches a but leaves b unconsumed. If the complete input must be consumed, parsing fails; PEG does not automatically retry 'ab' merely because the enclosing parse later fails.
Changing the order gives:
S <- 'ab' / 'a'
Now the first alternative matches the complete input ab.
Therefore, alternative order affects PEG parsing results. Backtracking restores an earlier input position when an alternative fails during its own evaluation.

6. Comparison with CFG-Based Parsers

Feature	LL Parser	LR Parser	PEG Parser
Parsing method	Top-down	Bottom-up	Top-down
Alternative selection	Predictive lookahead	Parsing table	Ordered choice
Left recursion	Generally removed	Supported	Requires special handling in conventional PEGs
Ambiguity	May require grammar changes	May produce conflicts	Ordered choice defines a prioritized interpretation
Backtracking	Usually limited	Usually unnecessary	Supported
Error recovery	Predictive recovery	State-based recovery	Requires suitable recovery rules
Implementation	Relatively simple	More complex	Simple for suitable grammars

Performance Analysis

Let (n) be the input length.

- **Time complexity:** (O(n)) for the specified expression grammar under a standard implementation.
- **Worst-case time:** Naive backtracking can take exponential time for some PEG grammars.
- **Space complexity:** (O(n)) for storing the syntax tree and parsing state in this example.
- **Packrat parsing:** Memoizes parsing results for grammar rules at input positions, providing linear-time parsing for a fixed PEG grammar, with typically (O(n)) memoization space.

Advantages and Limitations

Advantages

1. Ordered choice provides predictable alternative selection.
2. Grammar rules are readable and easy to organize.
3. Operator precedence is represented clearly.
4. Syntax trees can be constructed during parsing.
5. Packrat parsing can reduce repeated parsing work.

Limitations

1. Alternative ordering can affect parsing results.
2. Backtracking may increase execution time.
3. Packrat parsing consumes additional memory.
4. Left-recursive grammars require special handling.
5. Error recovery must be designed carefully.

Conclusion

The PEG-based parsing system successfully parses id + id * id and constructs the correct syntax tree. It uses ordered choice to determine how alternatives are selected and supports backtracking when necessary. Compared with CFG-based LL and LR parsers, PEG offers readable grammar rules and predictable prioritized choices, but its performance, memory use, and error recovery depend on the grammar and parsing strategy.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	20	Demonstrates thorough understanding and integrates multiple concepts effectively	Good understanding with proper integration of most concepts	Basic understanding with partial integration	Poor understanding with no integration
Design & Implementation	25	Well-designed solution with systematic and optimal implementation	Correct design with minor issues	Basic design with errors or inefficiencies	Incorrect or incomplete design
Parser/Algorithm Construction	20	Accurate construction of parser/algorithm with complete logic	Mostly correct construction with minor errors	Partial construction with missing logic	Incorrect or incomplete construction
Analysis & Validation	15	Insightful analysis with correct validation and justification	Good analysis with reasonable justification	Basic analysis with limited validation	Poor or incorrect analysis
Documentation & Presentation	20	Well-structured documentation with diagrams, steps, and clarity	Organized documentation with required details	Basic documentation with missing details	Poor or incomplete documentation